



2024

程序静态分析



## 程序静态分析概述



**程序静态分析**是指在不运行程序的前提下，仅通过分析或检查程序的语法、结构、过程、接口等对程序进行分析的过程，其主要目的在于检测软件中的缺陷



统计表明，在整个软件开发生命周期中，30%至70%的代码逻辑设计和编码缺陷是可以通过静态代码分析来发现和修复的。在程序静态分析过程中，要尽可能的发现代码中的逻辑错误，让代码符合正确逻辑，并进一步满足高效性、清晰性、规范性、一致性等要求



## 程序静态分析概述



图2-1 静态白盒测试示意图



程序静态分析通过分析源程序的语法、结构、过程和接口等来检查验证程序的正确性。其中，语法错误检查是最基本的程序静态分析工作之一，该检查通常由编译器完成

# 目录

## CONTENTS

### PART 01 静态分析概述

代码评审

结构分析

### PART 02 程序流程分析

控制流分析

数据流分析

### PART 03 符号运行

静态符号运行

动态符号运行

### PART 04 编程规约



## 程序静态分析概述



当程序中语法错误被修正后，研发人员还需借助人工或其它静态方法来进一步检测程序中的非语法类型缺陷，如不匹配的参数、不适当的循环嵌套和分支嵌套、不允许的递归、未使用过的变量、空指针的引用以及可疑计算等。其中，代码评审、静态结构分析是常用的程序静态分析方法



# 静态分析概述

# 目录

## CONTENTS

PART 01 静态分析概述

代码评审

结构分析

PART 02 程序流程分析

控制流分析

数据流分析

PART 03 符号运行

静态符号运行

动态符号运行

PART 04 编程规约



## 常见代码评审项目

| 代码评审项目            |                  |
|-------------------|------------------|
| • 所有设计要求是否都实现？    | • 每一功能目的是否都有注释？  |
| • 代码编制是否遵照编码规范？   | • 是否按注释类型格式编写注释？ |
| • 所有的代码是否风格保持一致？  | • 代码注释量是否达到了规定值？ |
| • 所有的注释是清楚和正确？    | • 所有变量的命名是否依照规则？ |
| • 所有代码异常处理是否都有注释？ | • 循环嵌套是否优化到最少？   |

表2-1 常见的代码评审项目



# 代码评审



代码评审法主要通过**桌面检查**、**代码审查**和**代码走查**等方式对软件代码进行检查



## 桌面检查

桌面检查是指开发人员通过阅读程序、对照错误列表、推演测试数据等方式对自己代码进行缺陷检测的方法，是一种常用的代码评审方法

# 代码评审



## 代码审查

代码审查是指由若干开发人员和测试人员组成的审查小组，通过阅读、讨论、评价和审议等，对程序进行静态分析的过程



代码审查通常分为两步：第1步，小组负责人把设计规格说明书、控制流程图、程序文本及有关要求和规范分发给小组成员，作为审查的依据；第2步，召开程序审查会，通过会议和集体讨论、评价和审议，以集体的智慧从不同的角度分析程序中的问题

# 代码评审



## 代码走查

代码走查是指由人扮演计算机的角色，把数据代入程序并模拟代码的运行，观察程序是否正常运行过程

代码走查过程也分为两步：第1步，小组负责人把材料先发给走查小组每个成员，让他们认真研究程序；第2步，组织代码走查会议。与代码审查不同，与会人员不仅需要阅读程序和对照错误检查表进行检查，还需要让与会者模拟代码的运行，共同分析检查程序的运行过程和结果



代码审查与代码走查等方法都以小组为单位进行，但与代码审查不同，代码走查着重于模拟程序的运行

# 目录

## CONTENTS

### PART 01 静态分析概述

代码评审

结构分析



### PART 02 程序流程分析

控制流分析

数据流分析

### PART 03 符号运行

静态符号运行

动态符号运行

### PART 04 编程规约



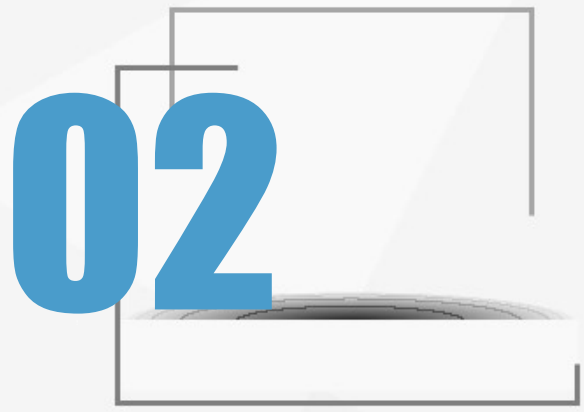
## 结构分析



图2-2 静态结构分析示意图



静态结构分析是帮助测试人员理解软件整体架构的有效方法



02

## 程序流程分析



## 程序流程分析



程序在模块运行次序、变量定义使用等方面需要满足一些要求来保证程序能正常运行。一般的，需要从程序的控制流和数据流等方面开展程序流分析

# 目录

## CONTENTS

### PART 01 静态分析概述

代码评审

结构分析

### PART 02 程序流程分析

控制流分析

数据流分析

### PART 03 符号运行

静态符号运行

动态符号运行

### PART 04 编程规约



# 控制流分析



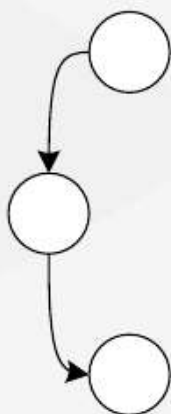
## 定义1. 控制流图CFG (Control Flow Graph)



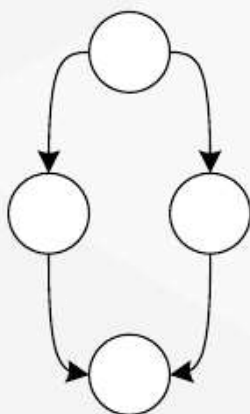
控制流图CFG= $\langle N, E, n_{entry}, n_{exit} \rangle$ 是一个有向图，用于描述程序的控制流程。其中， $N$ 是节点集，每个节点 $n$ 对应一条程序语句 $s$ ； $E$ 是边集，每条边 $e=\langle n_1, n_2 \rangle$ 表示语句 $s_1$ 运行后可能会立即运行语句 $s_2$ ； $n_{entry}$ 和 $n_{exit}$ 属于 $N$ ，分别表示程序唯一的入口节点和出口节点，且对于 $N$ 中其它节点 $n$ ，均至少存在一条由 $n_{entry}$ 经 $n$ 到达 $n_{exit}$ 的路径



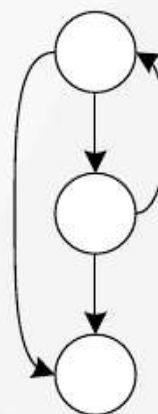
## 控制流分析



顺序结构



IF 选择结构



WHILE 循环结构



UNTIL 循环结构

图2-3 控制语句对应的控制流图



控制流分析的目的在于构造一个表达程序结构的控制流图



## 控制流相关缺陷及影响

| 缺陷        | 影响       |
|-----------|----------|
| 转向并不存在的标号 | 程序运行意外中止 |
| 存在无用的语句标号 | 占用额外管理资源 |
| 存在不可达语句标号 | 相应功能无法调用 |
| 不能到达可停机语句 | 程序运行难以中止 |

表2-3 控制流相关缺陷及影响

# 目录

## CONTENTS

### PART 01 静态分析概述

代码评审

结构分析

### PART 02 程序流程分析

控制流分析

数据流分析



### PART 03 符号运行

静态符号运行

动态符号运行

### PART 04 编程规约



# 数据流分析



**定义2. 变量定义DEF (Definition)**



变量 $v$ 存在于语句 $s$ 中，若 $s$ 运行时改变了 $v$ 的值，则称 $v$ 被 $s$ 定义



**定义3. 变量使用USE (Usage)**



变量 $v$ 存在于语句 $s$ 中，若 $s$ 运行时使用了 $v$ 的值，则称 $v$ 被 $s$ 使用

# 数据流分析

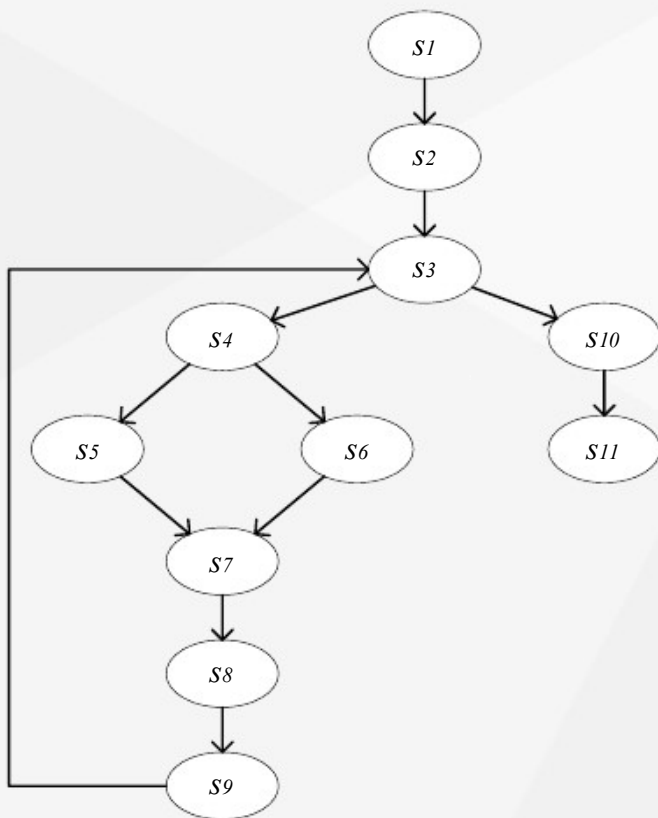


**数据流分析**是一种软件验证技术，用于分析变量在程序中的定义、使用及传递情况，来检测变量定义/使用错误和异常错误，主要包括以下三类：

- ① 变量被定义，但从未被使用
- ② 变量被使用，但还未被定义
- ③ 变量在使用之前被定义多次



## 数据流分析示例



(a) 控制流图

| 语句         | 定义变量           | 使用变量        |
|------------|----------------|-------------|
| <i>S1</i>  | <i>x, y, z</i> |             |
| <i>S2</i>  | <i>x</i>       | <i>w, x</i> |
| <i>S3</i>  |                | <i>x, y</i> |
| <i>S4</i>  |                | <i>y, z</i> |
| <i>S5</i>  | <i>y</i>       | <i>v, y</i> |
| <i>S6</i>  | <i>z</i>       | <i>v, z</i> |
| <i>S7</i>  | <i>v</i>       | <i>x</i>    |
| <i>S8</i>  | <i>w</i>       | <i>y</i>    |
| <i>S9</i>  | <i>z</i>       | <i>v</i>    |
| <i>S10</i> | <i>z</i>       | <i>z</i>    |
| <i>S11</i> |                | <i>z</i>    |

(b) 数据操作列表

图2-4 一个数据分析示例



## 数据流分析



通过数据流分析，可以发现该程序存在以下错误和异常：

- ① 语句s2使用了未定义的变量w
- ② 语句s5、s6首次运行时使用了未定义的变量v
- ③ 语句s8定义了变量w，但之后程序再未使用变量w
- ④ 语句s6、s9均定义了变量z，但两次定义之间并未再使用变量z



03

符号运行



## 符号运行概述



符号运行（**Symbolic Execution**）是一种程序分析技术，由King在1976年首次提出，目的在于得到运行特定区域代码的输入。在过去特别是近十年间，符号运行技术得到了研究人员的广泛关注



一方面，随着Z3、Yices、STP等功能强大的约束求解器的出现，符号运行技术可应用于规模更大、结构更复杂的真实程序中。另一方面，符号运行较其他程序分析方法计算代价更加昂贵，随着当前计算能力的显著提升，符号运行计算受限的问题得到了极大地缓解



## 符号运行概述

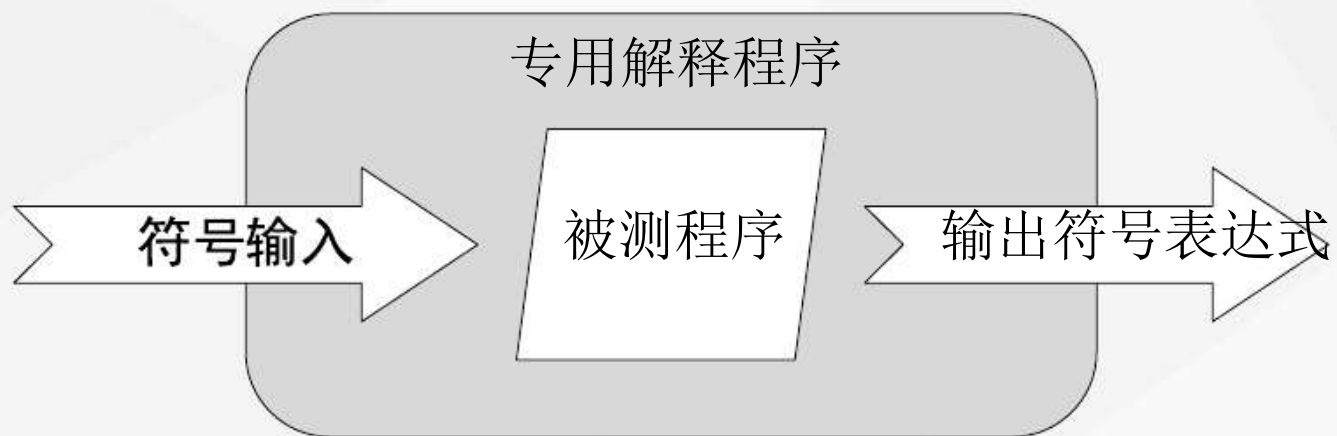


图2-5 符号运行概述



符号测试是一种介于程序运行与程序正确性证明之间的方法



## 符号运行概述



符号运行的作用主要体现在以下两点：其一是检查程序运行结果是否符合预期；其二是通过符号运行产生程序的运行路径，为进一步自动生成测试数据提供约束条件



根据符号运行过程中是否使用具体值，可将符号运行技术分为静态符号运行和动态符号运行

# 目录

## CONTENTS

### PART 01 静态分析概述

代码评审

结构分析

### PART 02 程序流程分析

控制流分析

数据流分析

### PART 03 符号运行

静态符号运行

动态符号运行

### PART 04 编程规约





## 静态符号运行



**静态符号运行**是指在不运行程序的前提下，以符号值作为输入，并通过符号运行模拟代码运行的过程。在静态符号运行时，不使用一般程序运行时使用的具体值作为输入。在符号运行过程中的任何运行点，符号化运行的程序状态包括程序变量在该点时的符号值、可以到达该点的路径条件PC（Path Condition）的符号值，以及一个程序计数器



## 静态符号运行



路径条件是一个建立在符号化输入上的布尔公式，是运行某条路径时输入必须满足的限制的积累



符号运行成功后，可以根据其生成的路径条件计算程序输入。生成的路径条件将被SMT约束求解器（Satisfiability Modulo Theories Solver）进行求解。约束求解器会试图找到路径条件的解。如果约束求解器可以发现一个解，则该解可以分配给对应的输入变量，构成一个程序输入，该输入可以运行指定的路径或语句



## 静态符号运行

|     |                            |
|-----|----------------------------|
| s0  | void calculate(int x, y) { |
| s1  | if (x>y) {                 |
| s2  | x = x+y;                   |
| s3  | y = x-y;                   |
| s4  | x = x-y;                   |
| s5  | if (x-y > 0) {             |
| s6  | // do something            |
| s7  | }                          |
| s8  | }                          |
| s9  | System.out.println(x);     |
| s10 | System.out.println(y);     |
| s11 | }                          |

图 2-6 一个示例程序



进一步，通过一个示例程序来说明静态符号运行过程。图2-6给出了一个示例程序，该程序包含了两个输入变量x和y





## 静态符号运行

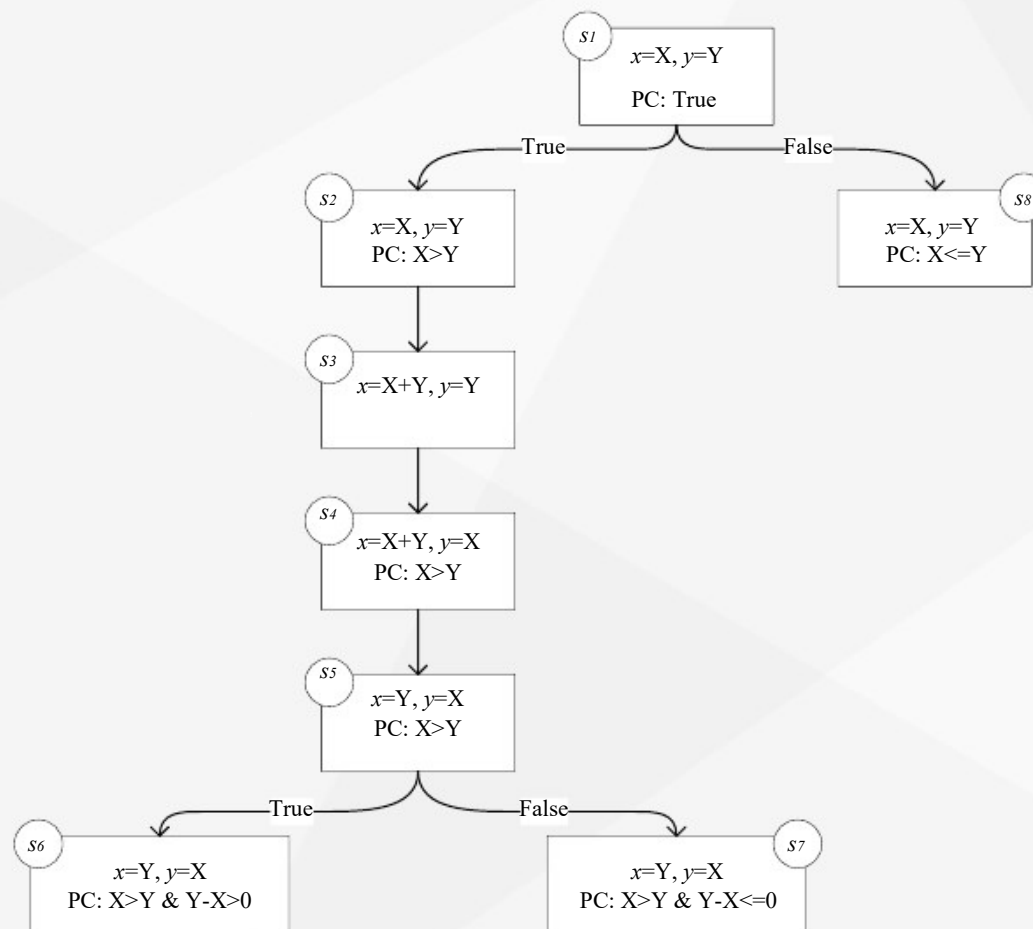


图 2-7 示例程序的符号运行树



## 静态符号运行

| 运行路径                                                                                         | 路径条件                            | 程序输入       |
|----------------------------------------------------------------------------------------------|---------------------------------|------------|
| path1: $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4 \rightarrow s_5 \rightarrow s_6$ | $(X > Y) \ \& \ (Y - X > 0)$    | None       |
| path2: $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4 \rightarrow s_5 \rightarrow s_7$ | $(X > Y) \ \& \ (Y - X \leq 0)$ | $X=2, Y=1$ |
| path3: $s_1 \rightarrow s_8$                                                                 | $X \leq Y$                      | $X=1, Y=1$ |

表 2-4 示例程序的路径条件及对应解



通过符号运行树，可以得出示例程序中的各条路径以及每条路径的路径条件。

表2-4给出示例程序各条路径的路径条件及对应解

# 目录

## CONTENTS

### PART 01 静态分析概述

代码评审

结构分析

### PART 02 程序流程分析

控制流分析

数据流分析

### PART 03 符号运行

静态符号运行

动态符号运行



### PART 04 编程规约



## 动态符号运行



尽管约束求解器性能的显著改进和计算能力的持续增强提高了符号运行解决实际问题的能力，但在处理大规模、复杂的程序时还存在较大的不足。例如，程序规模增加导致的路径爆炸问题，复杂的函数运算、浮点运算问题，以及复杂路径约束等问题，传统的符号运行技术就无法精确求解路径，也无法获得满足路径条件约束的测试用例。针对这些问题，研究人员提出了**动态符号运行**技术来进行解决



## 动态符号运行



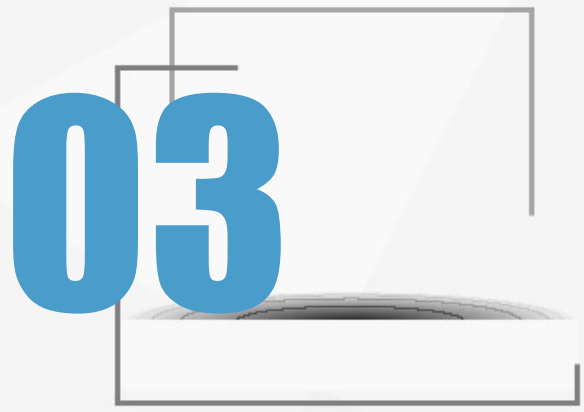
与静态符号运行不同，动态符号运行同时结合了符号输入和具体输入对程序进行分析。在符号输入的基础上，当遇到复杂路径或大规模路径时，使用具体输入值代替符号输入，以此来驱使符号运行继续向后开展，并可获得可求解的约束路径，提高了符号运行的效率。同时，由于使用具体输入值代替了部分符号输入，使得路径约束所包含的复杂数据结构和表达式得以简化，因而也大大减小了符号运行的代价



## 动态符号运行



在动态符号运行时，运行工具要跟踪记录符号状态以及当前路径的运行条件。在一条路径运行结束后，运行工具将路径中未覆盖分支的最后一个路径条件约束取反，再将新的路径条件传递给约束求解器进行求解。如果约束求解器可以给出一个满足新路径条件的解，运行工具会运行该条路径并重复上述过程，直至所有路径被覆盖，或覆盖特定目标，或满足时间需求

A stylized graphic of a book with a blue number 03 overlaid. The book is depicted with a white cover and a dark spine, and the number 03 is in a bold, blue font.

03

编程规约

# 编程规约



当前软件行业发展十分迅速，新的软件开发平台、工具和技术层出不穷，为软件研发人员带来便利的。同时，也使得更多的人可以同时参与到同一个项目中，共同完成规模更加庞大、结构更加复杂的软件系统。在这种形势下，如何有效地组织和管理庞大的研发生产队伍成为了一个巨大的挑战。



# 编程规约



编程是软件开发过程中的重要环节，在编码过程中，如果大家事先制订和遵循了统一的编码标准，则可以提高代码的可读性和可理解性，尽可能避开软件开发过程中可能存在错误 和遗漏 最终起到降低协成本、提升软件产品质量的效果



由于编程规范与规则的重要性，部分大型软件开发公司相继提出并开放了属于自己的编程标准，例如Google公司针对多种语言（包括Java、C++、Object-C等）提出了相应的编码规范，国内如阿里巴巴公司也提出了中英文版本的面向Java程序的开发手册，同时还提供了Java开发规约插件，用以帮助研发人员自动检测自己代码中违规的部分

# 编程规约



在开发者测试过程中，开发人员同样需要遵循一系列规范和规则，用于保证软件测试的正确性、高效性，以及测试代码的可阅读性、可理解性、可维护性和可扩展性



下面从全国大学生软件测试大赛开发者竞赛的角度，总结出4点规范和4点规则，用以指导选手编写更高质量的单元测试代码

# 编程规约



**规范(1): 命名格式。**测试类文件应当命名为: 类名+Test.java; 测试用例应当命名为: test+方法名。其中, 方法名的首字母大写

**说明:** 命名不规范会导致严重影响测试用例的可读性, 也不利于后期分析



**规范(2): 语句覆盖。**单元测试尽可能的覆盖所有语句

**说明:** 仅仅考虑分支覆盖会使得测试人员忽视没有包含分支语句的代码模块。  
因此, 参赛选手在比赛时也应当考虑增加语句覆盖

# 编程规约



**规范(3): 测试粒度。**对于单元测试，要保证测试粒度足够小，有助于精确定位问题。单测粒度至多是类级别，一般是方法级别

**说明：**只有测试粒度小才能在出错时尽快定位到出错位置。单测不负责检查跨类或者跨系统的交互逻辑，那是集成测试的领域



**规范(4): 注解标签。**推荐使用诸如@Test、@Before进行注解，不建议使用诸如@org.junit.Test、@org.junit.Before方式进行注解。

**说明：**显式调用org.junit没有意义

# 编程规约



**规则(1): 文件存放位置。**单元测试代码必须写在如下工程目录: `src/test/java`, 不允许写在`src`或其他目录下

**说明:** 评分及数据分析直接扫描该目录下的测试类文件, 若测试代码未包含在 `src/test/java` 中, 则认为参赛选手未提交测试代码, 记为零分



**规则(2): 测试用例独立性。**测试用例应当避免相互调用, 同时也应当与运行顺序无关

**说明:** 在不同配置下, 测试用例的运行顺序是不同的。若存在测试顺序依赖, 会导致开发人员耗费额外的经历用于环境配置

# 编程规约



**规则(3): 测试用例可重复性。**测试用例在每次运行时，其运行结果应当是不变的，不会受到外界环境的影响

**说明：**测试用例在设计时应当借助Mock等技术减轻其对外部环境的依赖，防止测试用例在持续集成环境中不可用



**规则(4): 测试用例全自动运行。**单元测试应该是全自动运行的，并且非交互式的。测试框架通常是定期运行的，运行过程必须完全自动化才有意义

**说明：**输出结果需要人工检查的测试不是一个好的单元测试。单元测试中不准使用System.out来进行人工验证，必须使用assert语句来验证

谢谢欣赏